ENAE464 - 0202

Lab04: Particle Imaging Velocimetry

Due on April 3rd, 2026 at 11:59 PM

Dr. Silbaugh, 02:00 PM

Mikołaj Kostrzewa & Vai Srivastava

Experiment Performed: March 27th, 2026
Report Submitted: April 3rd, 2026

April 3rd, 2026

Enclosed is the technical report for the *Particle Imaging Velocimetry* laboratory experiment. This report presents the experimental methodology, results, and analysis of flow characteristics and imaging effects for a P.I.V. technique on the wake downstream of a circular cylinder.
Please feel free to contact us with any questions regarding the contents of this report.

Respectfully,
Mikołaj Kostrzewa & Vai Srivastava

Contents

1 Abstract **1**

2 Introduction **1**
2.1 Theoretical Background 1
2.2 Objectives 1

3 Methodology **1**
3.1 Experimental Apparatus 1
3.2 Operating Procedure 2

4 Results **2**
4.1 Flow Velocity 2
4.2 Velocity Fluctuation 2
4.3 Shedding Frequency 2
4.4 Stokes Number 2

5 Analysis and Discussion **2**
5.1 Shedding Frequency 2
5.2 Comparison to Theory 2

6 Summary and Conclusions **2**

7 References **2**

8 Appendices **2**
8.1 Data 2
8.2 Code 2

Figures

3.1.1 foo 1

Tables

Listings

8.2.1 Index File 2

1 Abstract

2 Introduction

2.1 Theoretical Background

2.2 Objectives

The specific objectives of this experiment are:

1. foo

3 Methodology

3.1 Experimental Apparatus

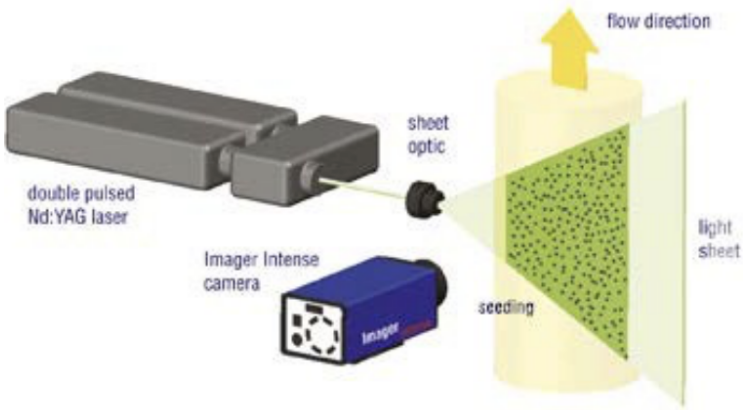

Figure 3.1.1: foo

3.2 Operating Procedure

4 Results

4.1 Flow Velocity

4.2 Velocity Fluctuation

4.3 Shedding Frequency

4.4 Stokes Number

5 Analysis and Discussion

5.1 Shedding Frequency

5.2 Comparison to Theory

6 Summary and Conclusions

7 References

[1] J. D. Anderson and C. P. Cadou, *Fundamentals of Aerodynamics*, 7th ed. McGraw-Hill Education, March 17, 2023, ISBN: 9781266076442.

8 Appendices

8.1 Data

For the sake of brevity, the raw measurement data will not be displayed in this report. However, in the spirit of completeness, the repository containing the complete data, source code, and notes for this report can be found at github:vaisriv/enae464-lab04.

8.2 Code

Similarly, only the code files that are key to the analysis are included below, and can be accessed in full in the aforementioned github repository.

Listing 8.2.1: Index File
`./src/index.py`

```python
# FIXME:
# scaffold enae464-lab04

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
from pathlib import Path

# ── Paths ──────────────────────────────────────────────────────────────
```

```python

DATA_DIR = Path("./data")
OUTPUT_TEXT_DIR = Path("./outputs/text")
OUTPUT_FIGS_DIR = Path("./outputs/figures")

OUTPUT_TEXT_DIR.mkdir(parents=True, exist_ok=True)
OUTPUT_FIGS_DIR.mkdir(parents=True, exist_ok=True)

# ── Constants ──────────────────────────────────────────────────────────

P_ATMOS = 102510.0  # Pa  (1025.1 hPa from Table 1)
T_ATMOS = 295.15  # K   (22°C from Table 1)
R_AIR = 287.05  # J/(kg·K)
RHO_ATMOS = P_ATMOS / (R_AIR * T_ATMOS)  # ≈ 1.210 kg/m³

GAMMA = 1.4
D1 = 0.024  # m  (inlet diameter)
D2 = 0.0095  # m  (throat diameter)
A_INLET = np.pi * D1**2 / 4
A_THROAT = np.pi * D2**2 / 4

# ── Load raw data ──────────────────────────────────────────────────────
# Columns as recorded from the C1-MKII console:
#   P1 [kPa]  =  p_atmos - p_throat
#   P2 [kPa]  =  p_atmos - p_outlet
#   P3 [Pa]   =  p_atmos - p_inlet

raw = pd.read_csv(
    DATA_DIR / "pressure_taps.csv", names=["P1_kPa", "P2_kPa", "P3_Pa"], header=0
)

# Convert gauge readings to absolute pressures [Pa]
p_throat = P_ATMOS - raw["P1_kPa"] * 1e3
p_outlet = P_ATMOS - raw["P2_kPa"] * 1e3
p_inlet = P_ATMOS - raw["P3_Pa"]

# ── Build relative-pressure table ──────────────────────────────────────

rel = pd.DataFrame(
    {
        "p_atmos [Pa]": P_ATMOS,
        "p_inlet [Pa]": p_inlet.round(4),
        "p_throat [Pa]": p_throat.round(4),
        "p_outlet [Pa]": p_outlet.round(4),
        "p_throat / p_atmos [-]": (p_throat / P_ATMOS).round(6),
    }
)

```

```python
rel.to_csv(OUTPUT_TEXT_DIR / "pressures.csv", index=False)
print(rel.to_string(index=False))
print(f"\nSaved → ./{OUTPUT_TEXT_DIR / 'pressures.csv'}")

# ── Mass flow rate ──────────────────────────────────────────────────────
# From the isentropic inlet flow equation:
#   ṁ = a_inlet · √( 2 · p_atmos · (p_atmos - p_inlet) )
#     = a_inlet · √( 2 · p_atmos · P3 )

mdot = A_INLET * np.sqrt(2 * RHO_ATMOS * raw["P3_Pa"])

# ── Build mass flow vs. pressures table ─────────────────────────────────

dp_outlet = raw["P2_kPa"] * 1e3  # p_atmos - p_outlet [Pa]
dp_throat = raw["P1_kPa"] * 1e3  # p_atmos - p_throat [Pa]

mvp = pd.DataFrame(
    {
        "p_atmos - p_outlet [Pa]": dp_outlet.round(4),
        "p_atmos - p_throat [Pa]": dp_throat.round(4),
        "p_atmos - p_inlet [Pa]": raw["P3_Pa"].round(4),
        "p_throat / p_atmos [-]": (p_throat / P_ATMOS).round(6),
        "mdot [kg/s]": mdot.round(6),
    }
)

mvp.to_csv(OUTPUT_TEXT_DIR / "mass_flow_vs_pressures.csv", index=False)
print(f"\n{mvp.to_string(index=False)}")
print(f"\nSaved → ./{OUTPUT_TEXT_DIR / 'mass_flow_vs_pressures.csv'}")

# ── Theoretical choked limits ───────────────────────────────────────────
# Critical pressure ratio: r_c = (2 / (γ+1))^(γ/(γ-1))
# Choked ṁ via the throat form of the isentropic mass flow equation

r_c = (2 / (GAMMA + 1)) ** (GAMMA / (GAMMA - 1))
mdot_max = (
    RHO_ATMOS
    * A_THROAT
    * np.sqrt(
        GAMMA * (P_ATMOS / RHO_ATMOS) * (2 / (GAMMA + 1)) ** ((GAMMA + 1) / (GAMMA - 1))
    )
)
dp_throat_choked = (1 - r_c) * P_ATMOS  # p_atmos - p_throat at choking

# ── Plot 1: ṁ vs. (p_atmos - p_outlet) ──────────────────────────────────

fig, ax = plt.subplots(figsize=(7, 4.5))

```

```python
ax.plot(
    dp_outlet / 1e3,
    mdot * 1e3,
    marker="o",
    linewidth=1.5,
    markersize=5,
    color="steelblue",
    label="Experimental",
)

ax.axhline(
    mdot_max * 1e3,
    color="tomato",
    linewidth=1.5,
    linestyle="--",
    label=rf"Theoretical $\dot{{m}}_\mathrm{{max}}$ = {mdot_max * 1e3:.3f} g/s",
)

ax.set_xlabel(r"$p_\mathrm{atmos} - p_\mathrm{outlet}$ [kPa]")
ax.set_ylabel(r"$\dot{m}$ [g/s]")
ax.set_title(r"Mass Flow Rate vs. Outlet Pressure Drop")
ax.xaxis.set_minor_locator(ticker.AutoMinorLocator())
ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())
ax.grid(which="major", linestyle="--", alpha=0.4)
ax.grid(which="minor", linestyle=":", alpha=0.2)
ax.legend()
fig.tight_layout()
fig.savefig(OUTPUT_FIGS_DIR / "mdot_vs_dp_outlet.png", dpi=200)
plt.close(fig)
print(f"\nSaved → ./{OUTPUT_FIGS_DIR / 'mdot_vs_dp_outlet.png'}")

# ── Plot 2: (p_atmos - p_throat) vs. (p_atmos - p_outlet) ──────────────────

fig, ax = plt.subplots(figsize=(7, 4.5))

ax.plot(
    dp_outlet / 1e3,
    dp_throat / 1e3,
    marker="o",
    linewidth=1.5,
    markersize=5,
    color="steelblue",
    label="Experimental",
)

ax.axhline(
    dp_throat_choked / 1e3,
    color="tomato",
```

```python
    linewidth=1.5,
    linestyle="--",
    label=rf"Theoretical choked throat $\Delta p$ = {dp_throat_choked / 1e3:.3f} kPa"
    rf"  $(r_c = {r_c:.3f})$",
)

ax.set_xlabel(r"$p_\mathrm{atmos} - p_\mathrm{outlet}$ [kPa]")
ax.set_ylabel(r"$p_\mathrm{atmos} - p_\mathrm{throat}$ [kPa]")
ax.set_title(r"Throat Pressure Drop vs. Outlet Pressure Drop")
ax.xaxis.set_minor_locator(ticker.AutoMinorLocator())
ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())
ax.grid(which="major", linestyle="--", alpha=0.4)
ax.grid(which="minor", linestyle=":", alpha=0.2)
ax.legend()
fig.tight_layout()
fig.savefig(OUTPUT_FIGS_DIR / "dp_throat_vs_dp_outlet.png", dpi=200)
plt.close(fig)
print(f"\nSaved → ./{OUTPUT_FIGS_DIR / 'dp_throat_vs_dp_outlet.png'}")
```